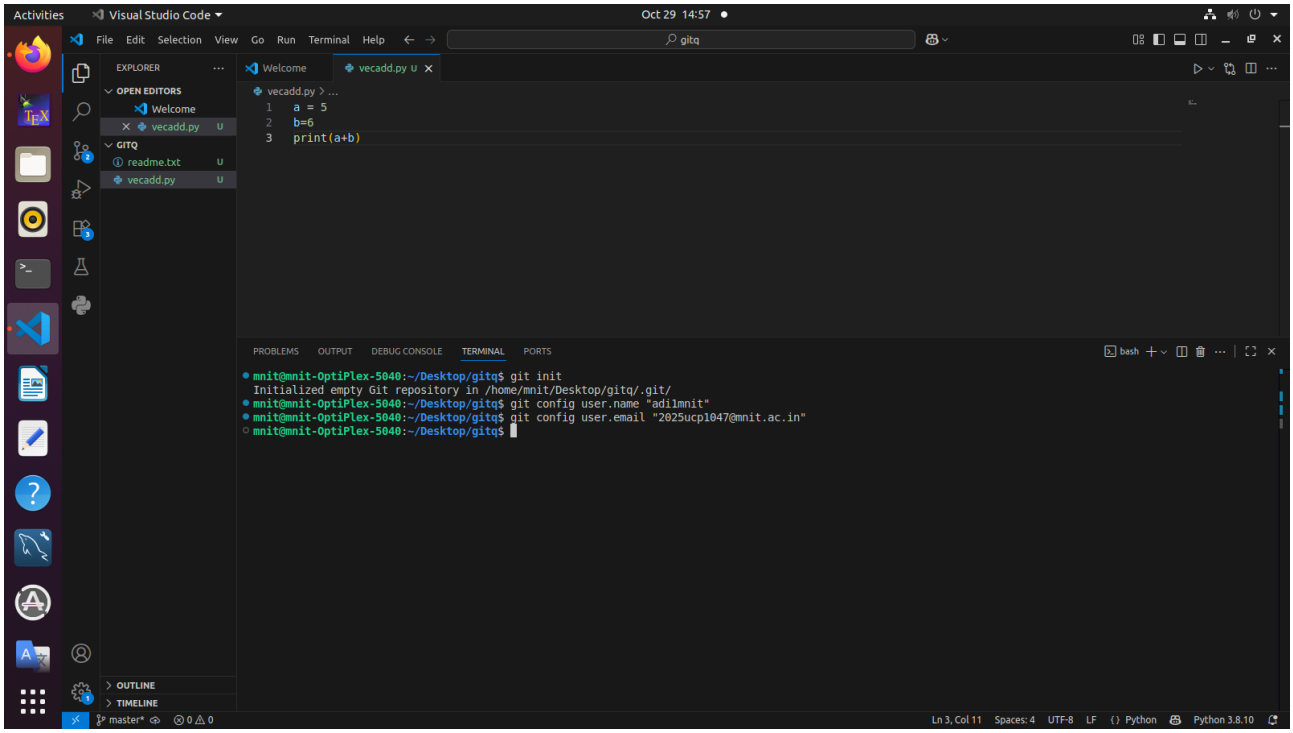
Assignment 8 - Git Basics

TW Lab 2025

Step 1: Initialise a Git repository

I created the project folder **gitq** on the Desktop, opened it in VS Code and ran `git init`. Git replied that it initialised an empty repository in *~/Desktop/gitq/.git/*. The folder also holds **vecadd.py**, a small Python script (a = 5, b = 6, print(a+b)).

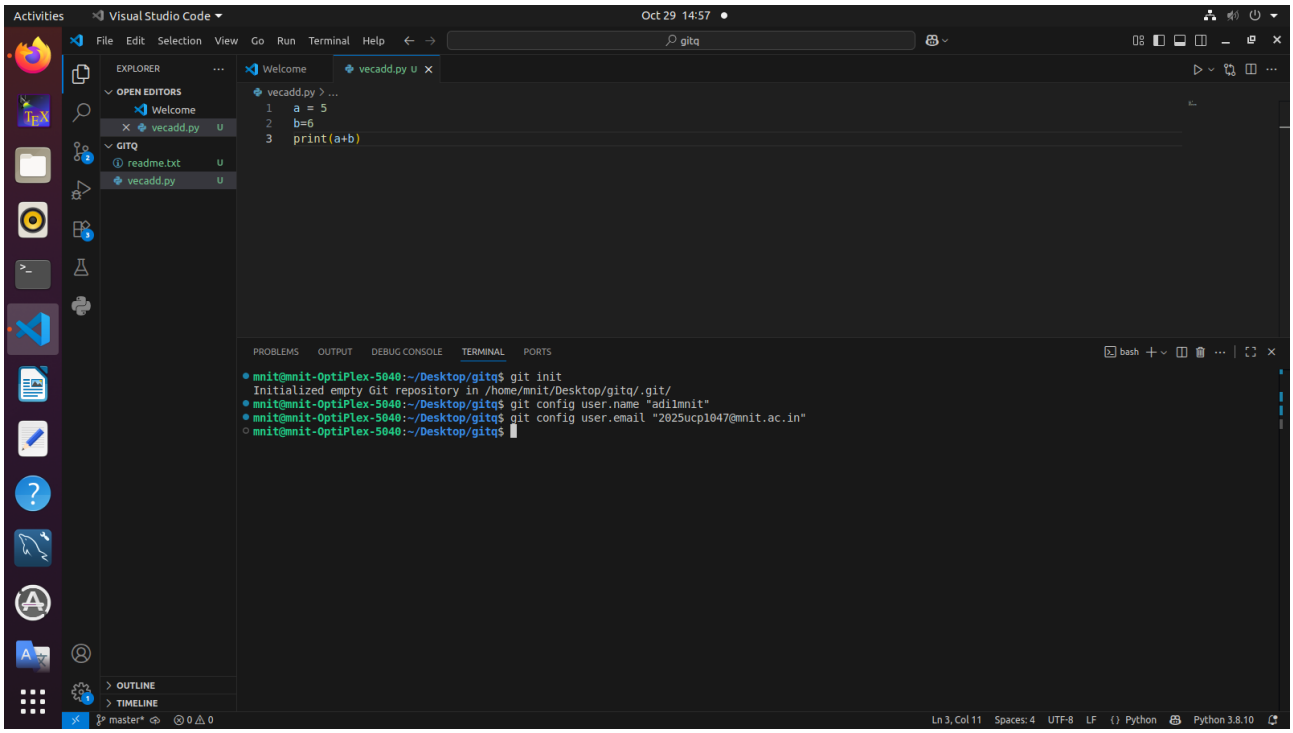
Screenshot: git init in the gitq folder.



Step 2: Configure name and email

I set the identity that Git attaches to every commit using `git config user.name` and `git config user.email`. These commands print nothing when they succeed.

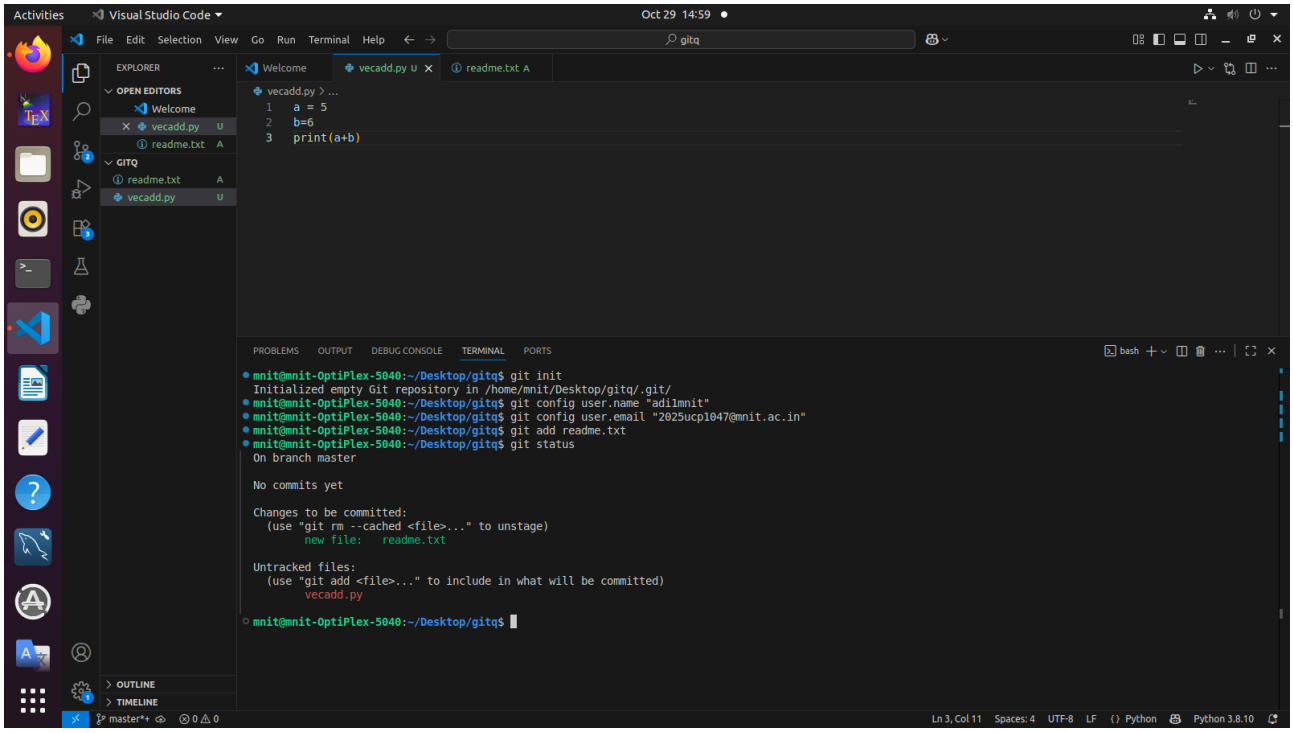
Same screenshot as above: the two git config commands appear right after git init.



Step 3: Create readme.txt, stage it and check status

I created **readme.txt**, staged it with `git add readme.txt` and ran `git status`. The output shows **readme.txt** under *Changes to be committed* (new file), while **vecadd.py** is listed as *Untracked* because it has not been added.

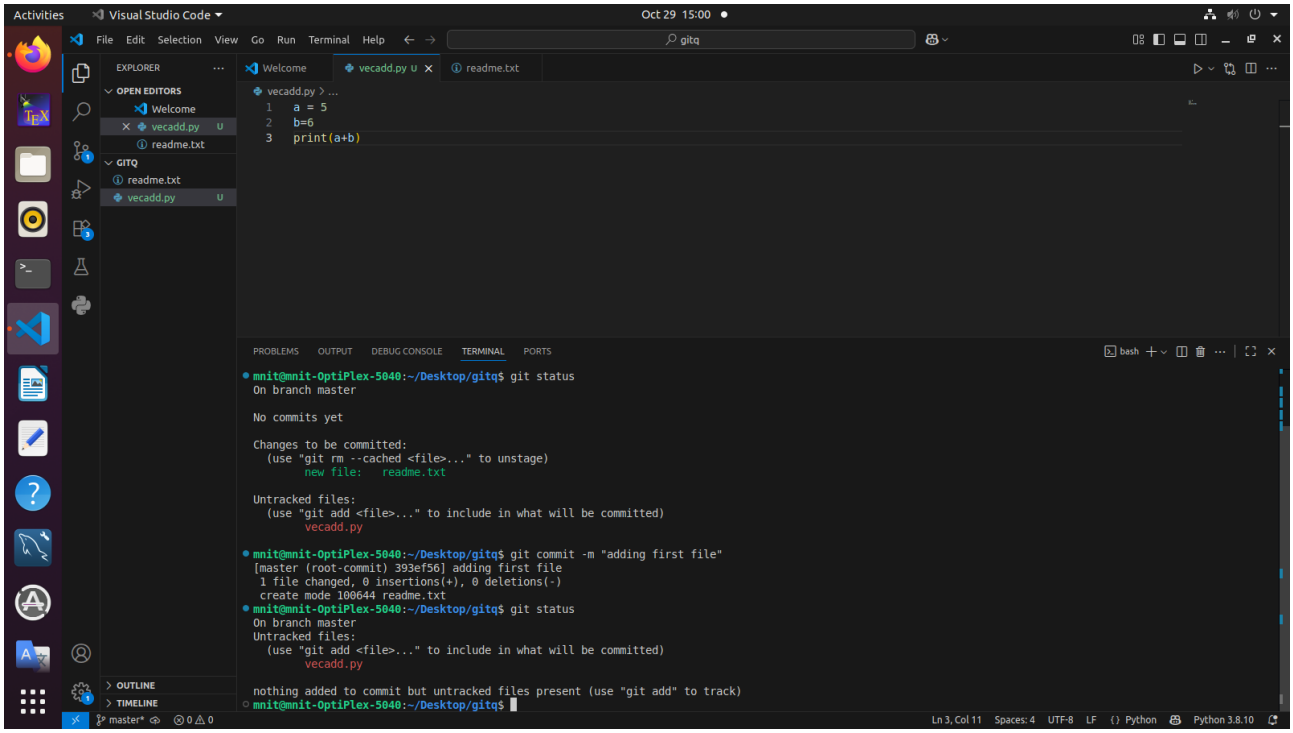
Screenshot: git add readme.txt followed by git status.



Step 4: Commit and verify a clean state

I committed with `git commit -m "adding first file"`. Git created the root commit **393ef56** (1 file changed, new file readme.txt). Running `git status` again no longer lists readme.txt, so it is now tracked; only the untracked vecadd.py remains.

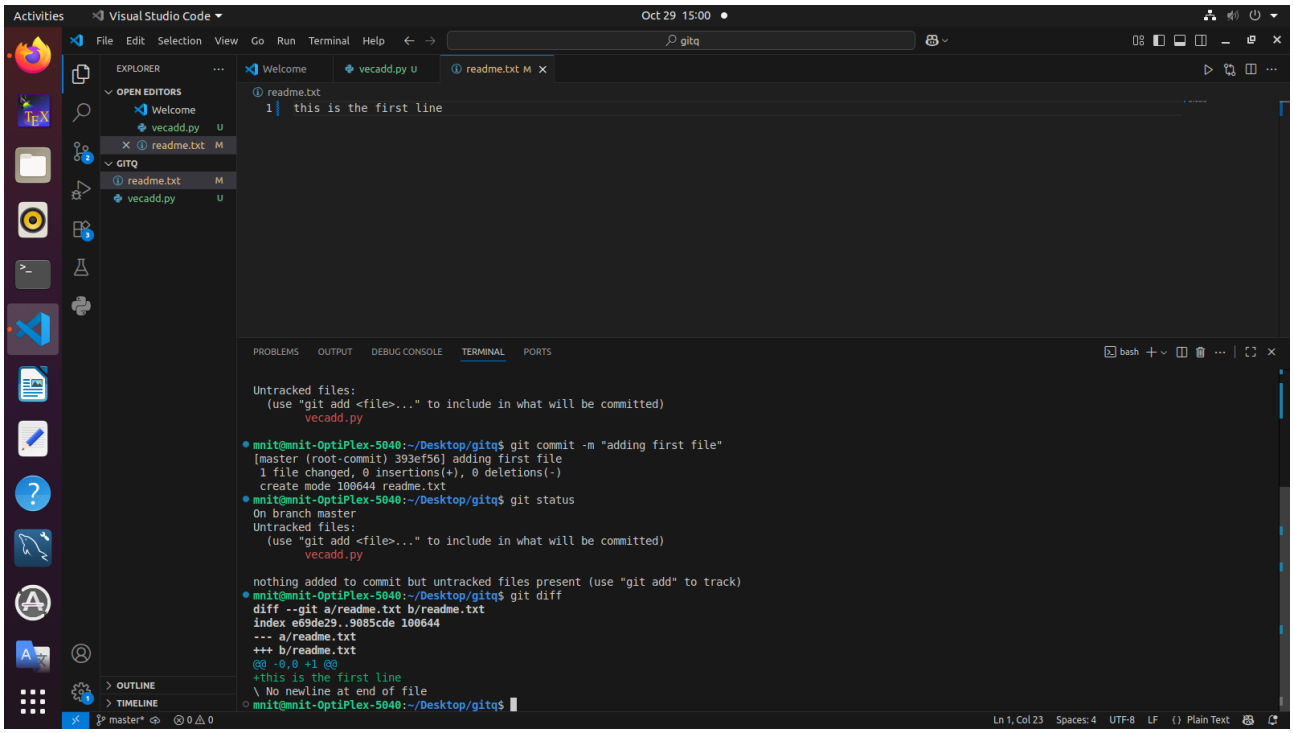
Screenshot: git commit, then git status after the commit.



Step 5: Modify readme.txt and view changes with git diff

I opened readme.txt and added the line *"this is the first line"*. `git diff` shows the change as a green **+** line (an addition) compared with the committed version, and notes that the file has no newline at the end.

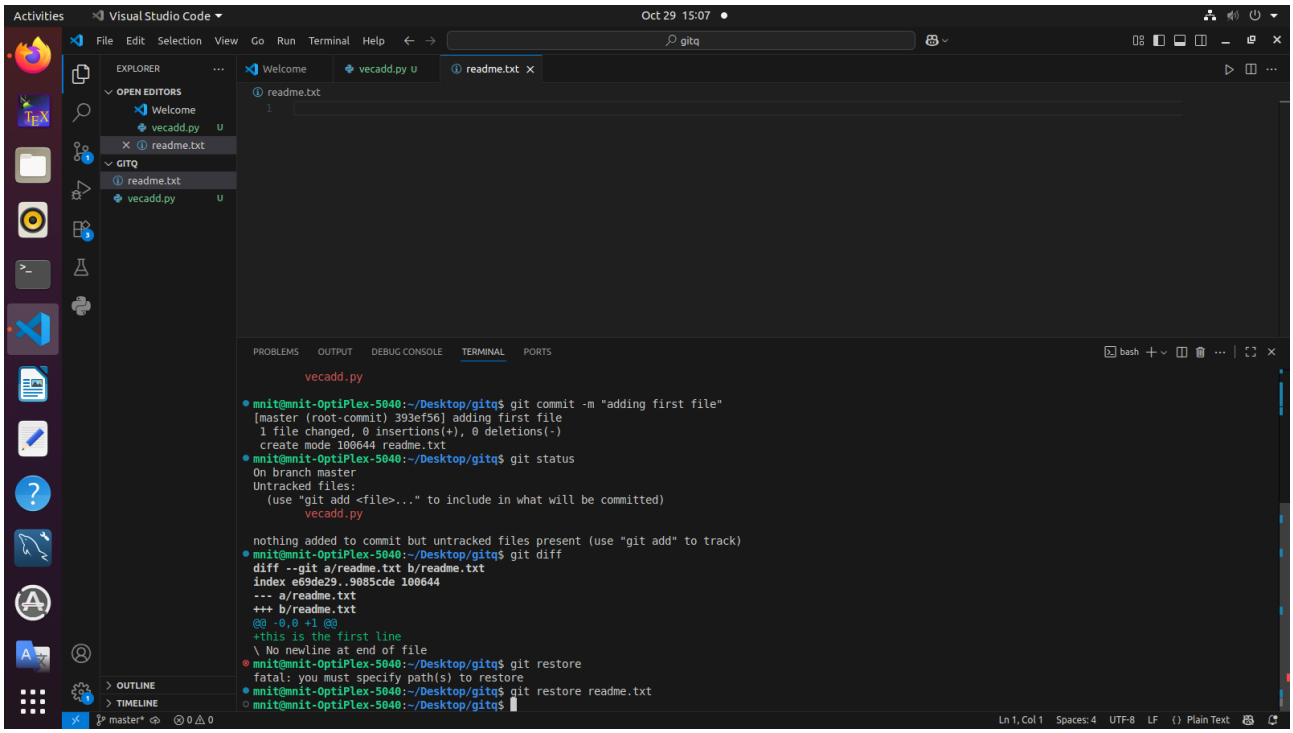
Screenshot: readme.txt edited in VS Code and the output of git diff.



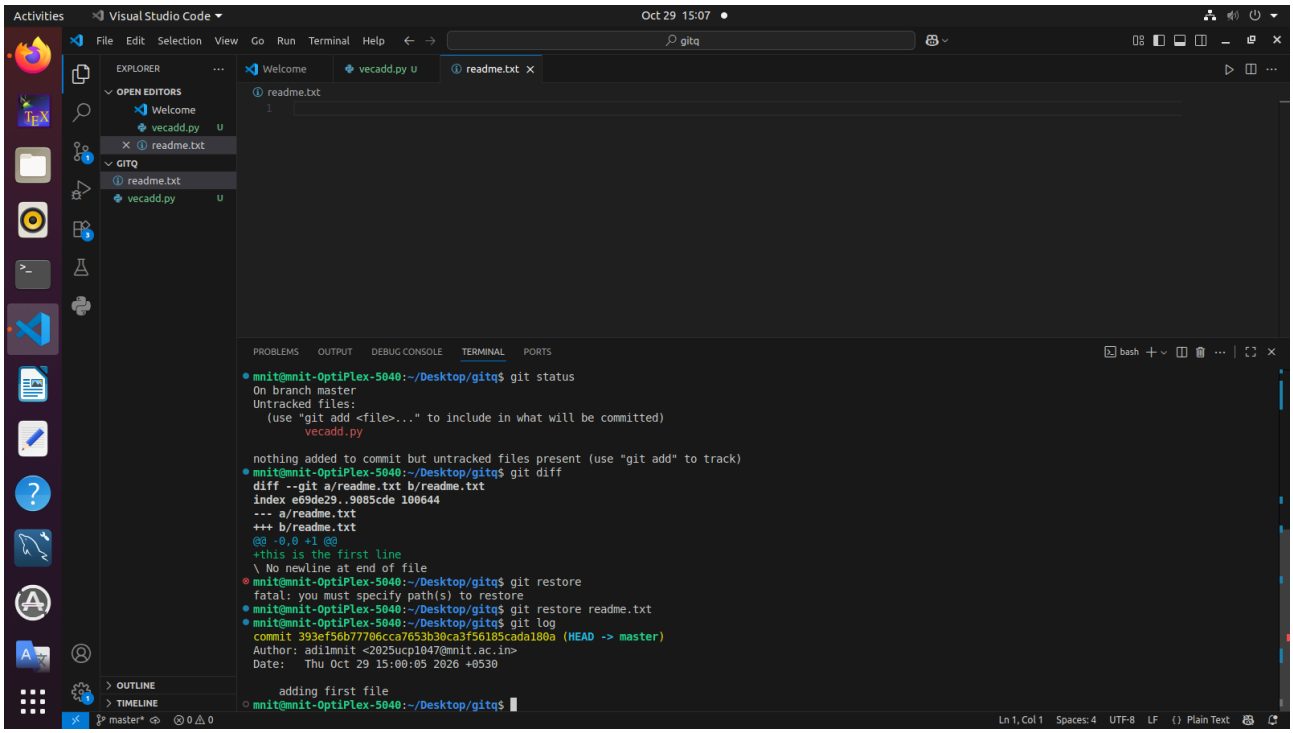
Step 6: Explore git restore, then commit again

`git restore` on its own gives *"fatal: you must specify path(s) to restore"*. Running `git restore readme.txt` on the **modified** file discarded my edit, so the file went back to its committed (empty) state and git diff shows nothing. After that I re-added a line to readme.txt (*"now readding this line and commiting"*), staged it with git add and committed it again with the message *"this is the second time"* (commit **e12ede6**).

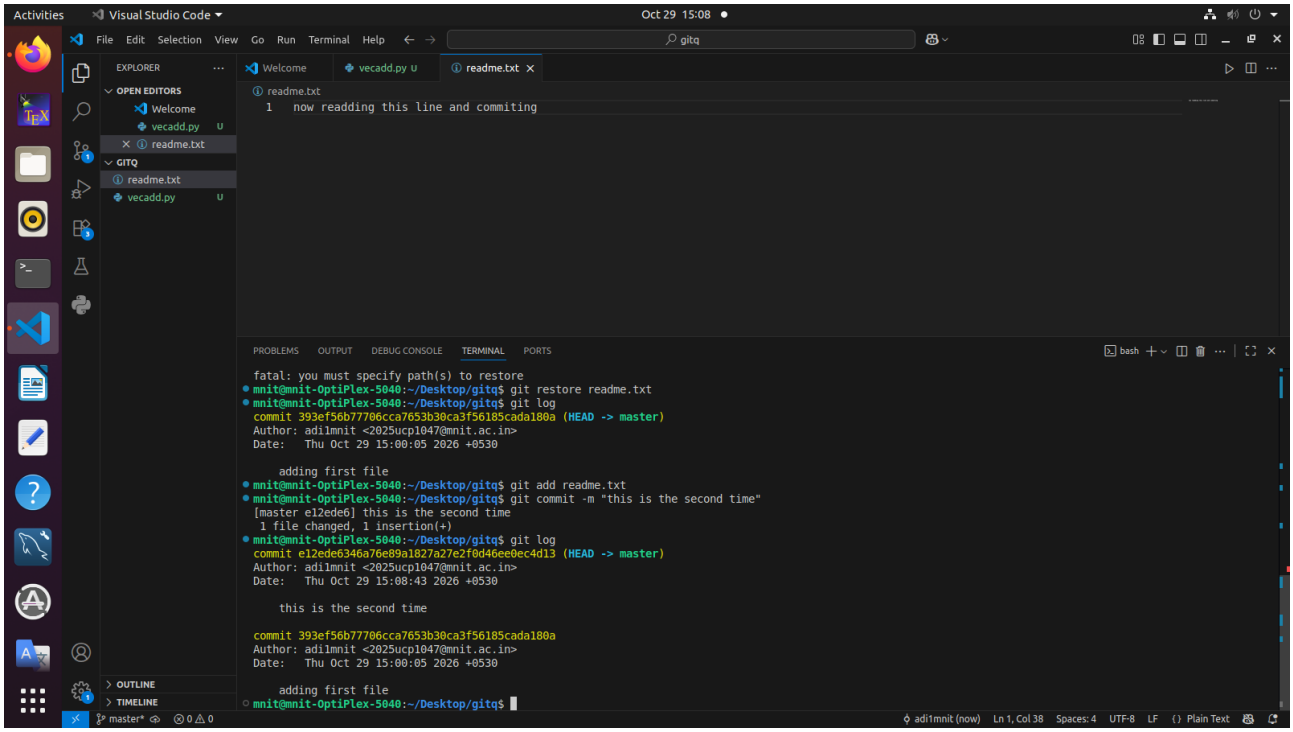
Screenshot: git restore without a path (error) and git restore readme.txt, which reverts the file.



Screenshot: git status/diff after the restore (clean) and git log.



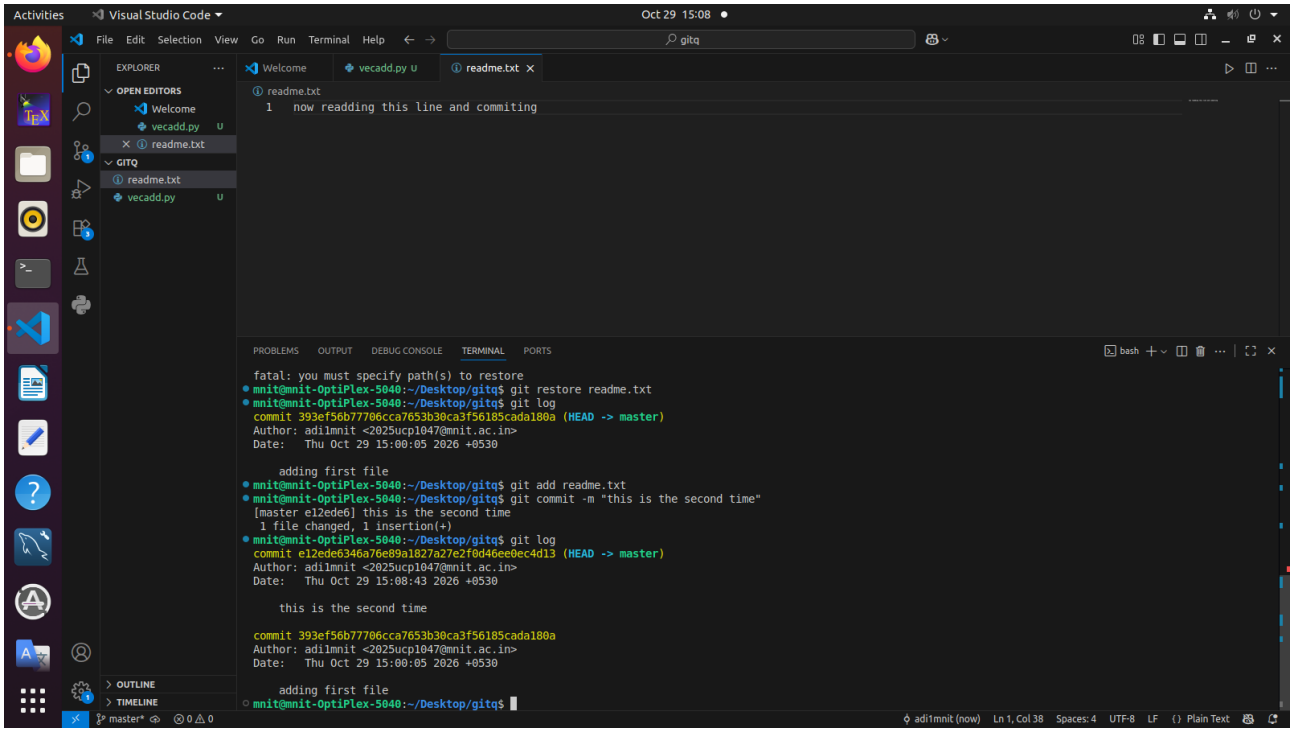
Screenshot: the new line in readme.txt, git add, the second commit and the log.



Step 7: View the history with git log

`git log` lists the commits newest first with their hash, author, date and message. At this point there are two commits: **e12ede6** "this is the second time" on top of **393ef56** "adding first file", and HEAD points to the newest one on **master**.

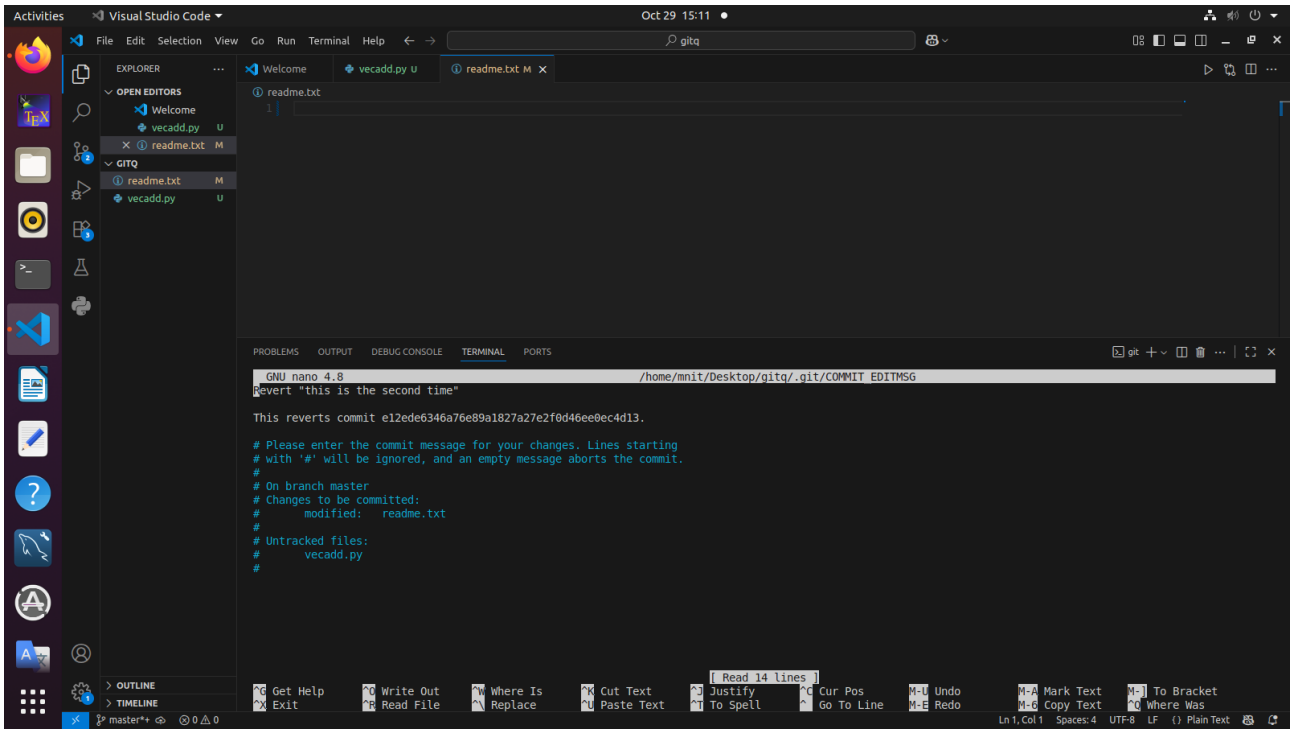
Screenshot: git log showing both commits (same screenshot as the end of Step 6).



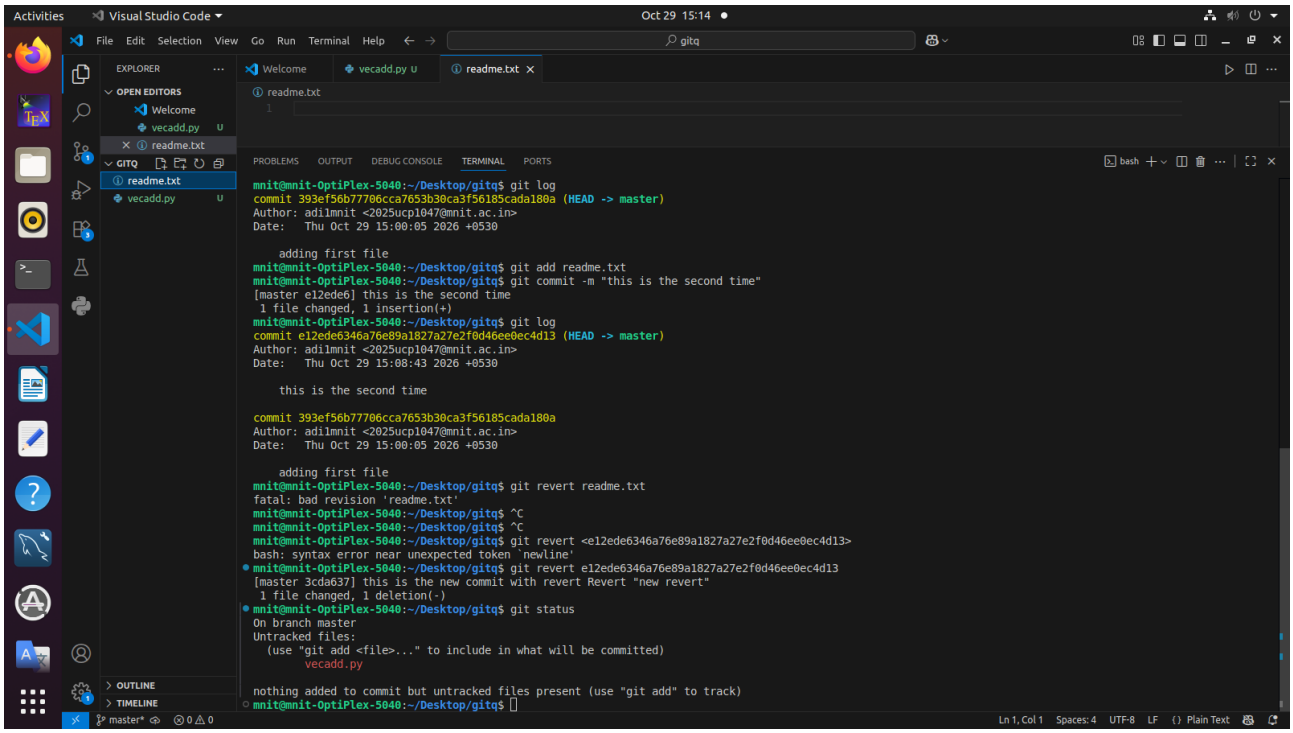
Step 8: Undo a commit with git revert

`git revert readme.txt` failed with *"bad revision"* because revert needs a commit hash, not a file name. Using the hash `git revert e12ede6346a76e89a1827a27e2f0d46ee0ec4d13` opened the nano editor with the default message *Revert "this is the second time"*. After saving, Git made a **new commit (3cda637)** that undoes the changes of e12ede6. `git log` shows that the old commit is **still in the history**; revert adds a commit on top instead of deleting anything.

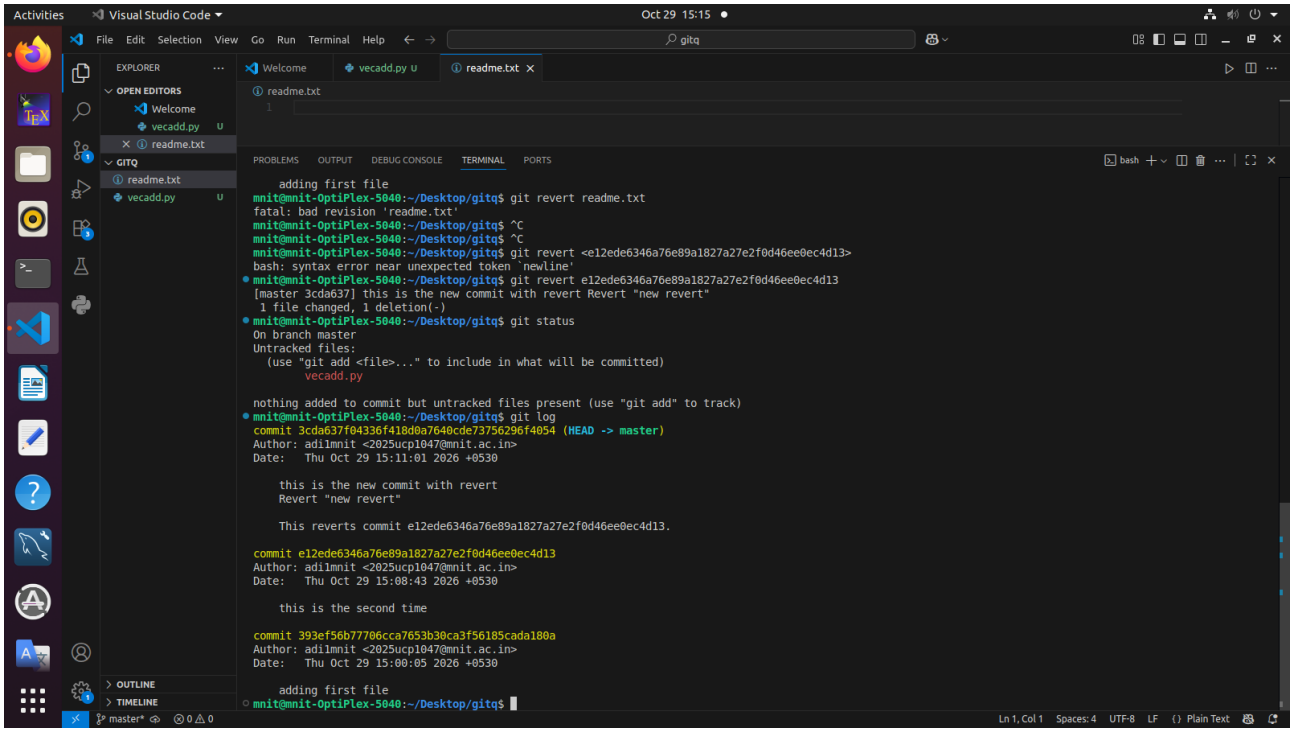
Screenshot: nano commit-message editor opened by git revert.



Screenshot: the bad-revision error, the working revert command and git status.



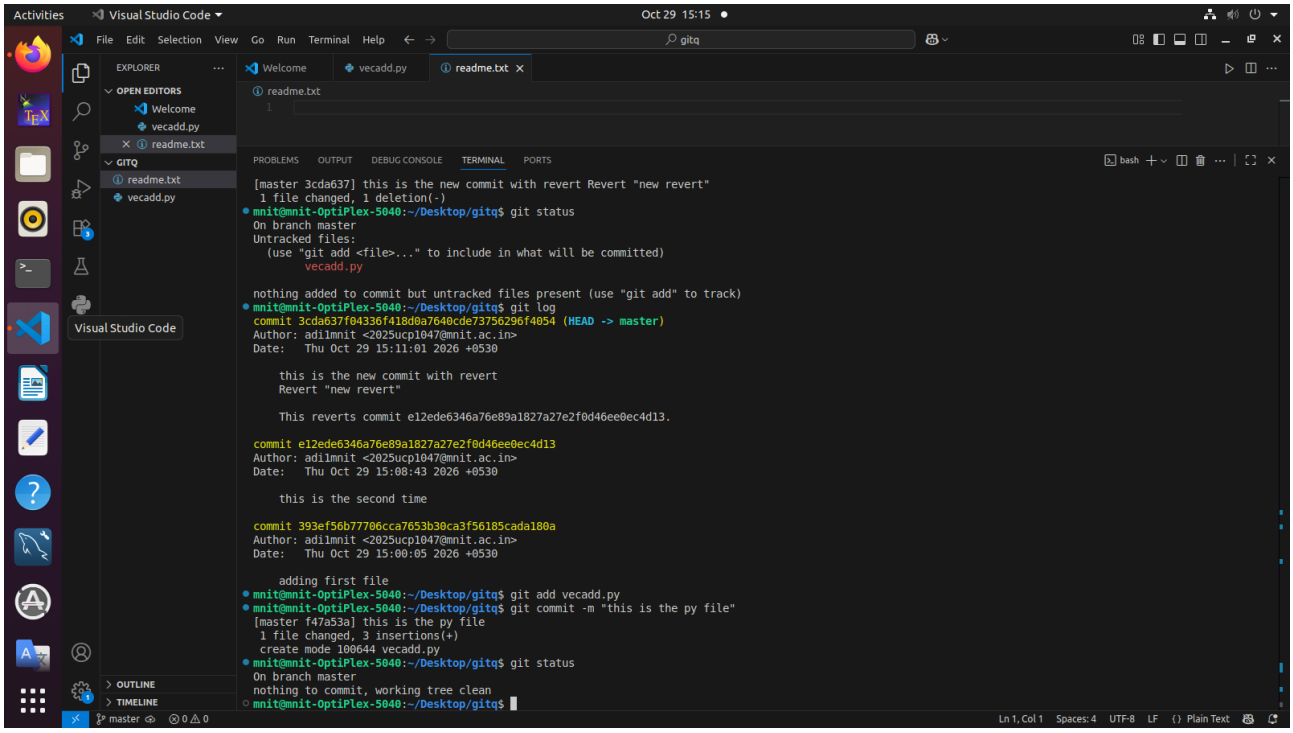
Screenshot: git status and git log showing the new revert commit above the old ones.



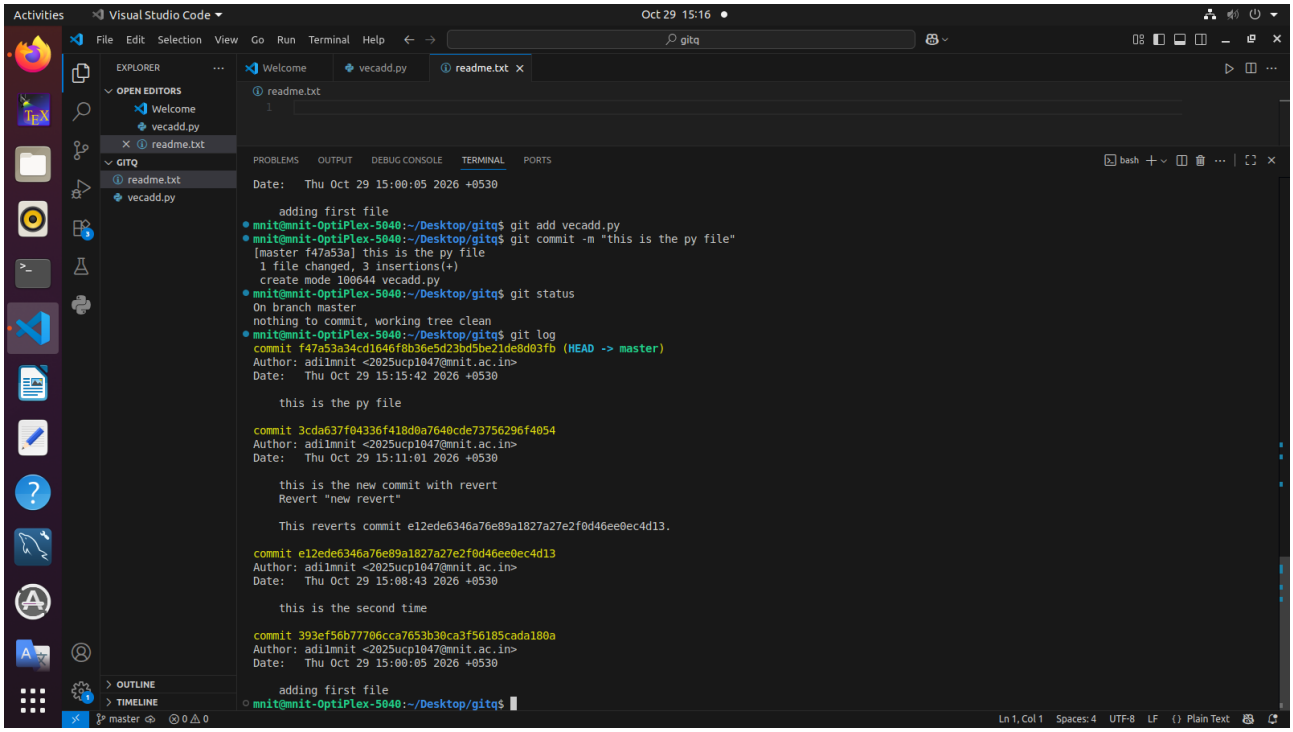
Step 9: Commit vecadd.py and show git log

I staged and committed the Python file with `git add vecadd.py` and `git commit -m "this is the py file"` (commit **f47a53a**, 3 insertions). `git status` reports *nothing to commit, working tree clean* and git log now shows four commits.

Screenshot: git add, git commit and git status for vecadd.py.



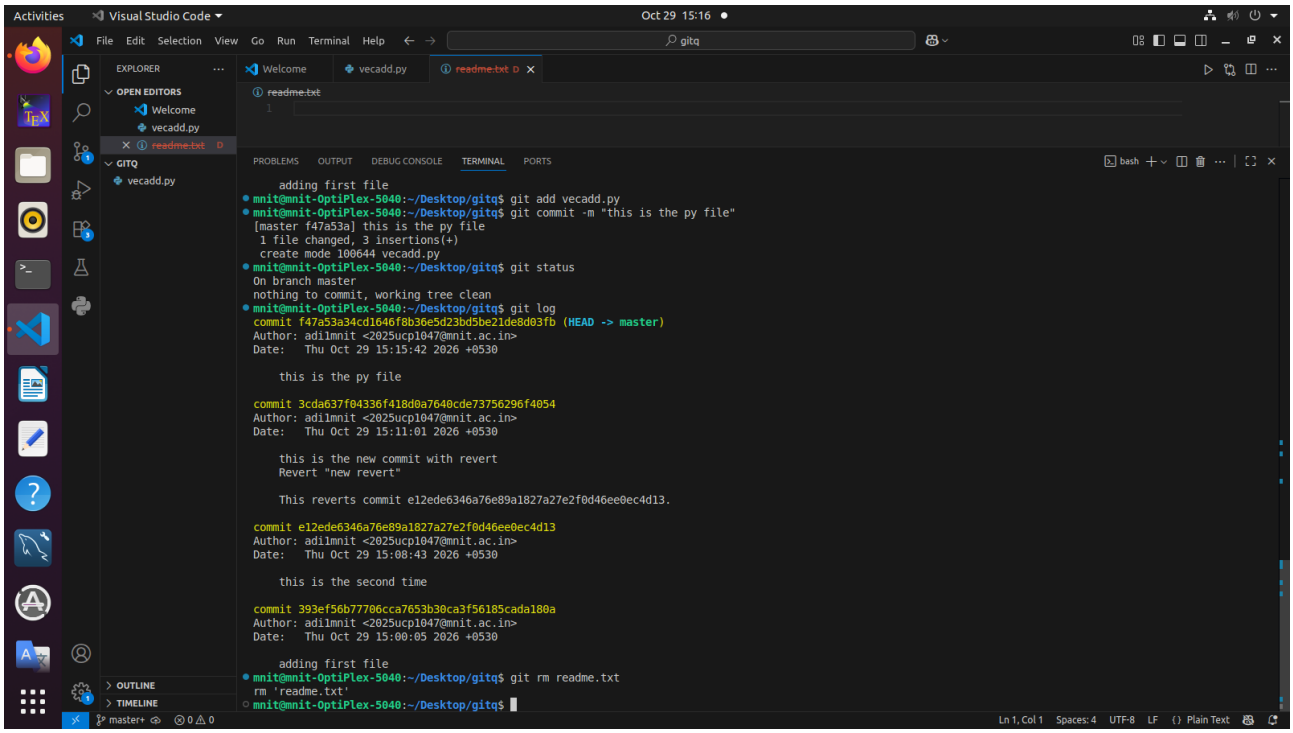
Screenshot: git log after committing vecadd.py.



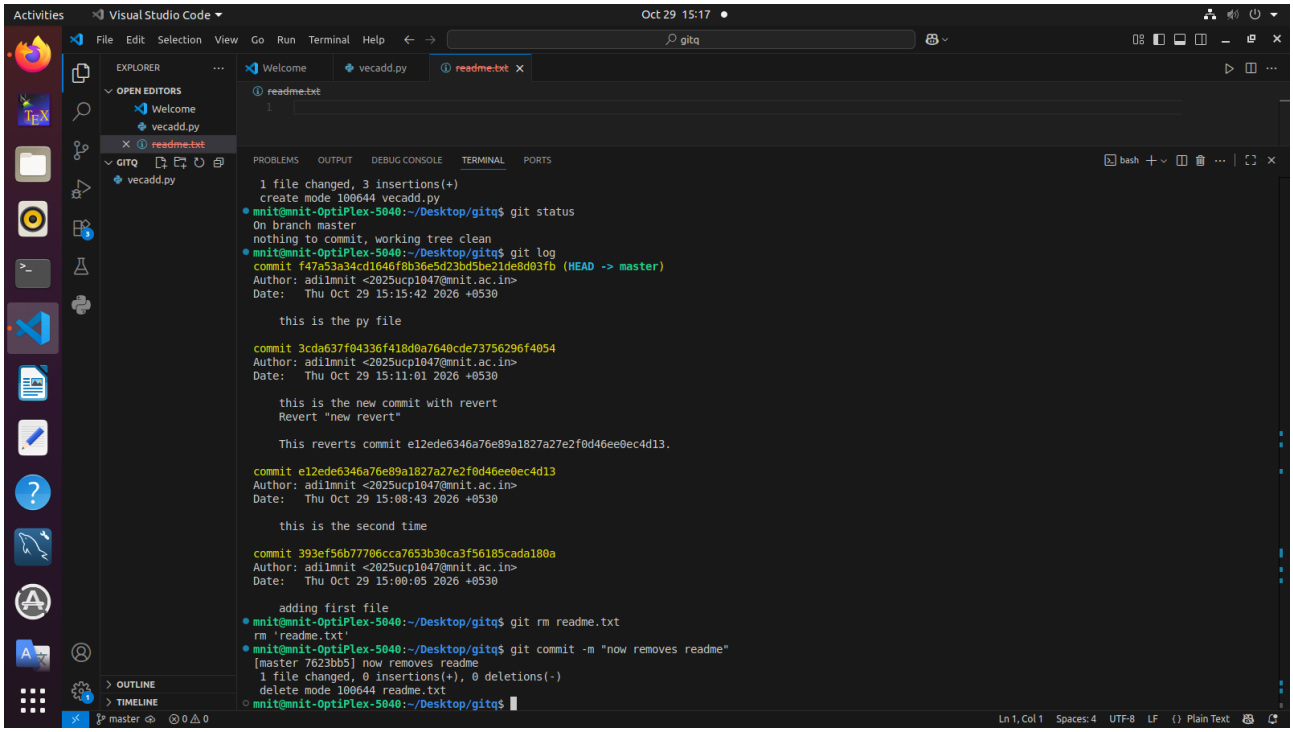
Step 10: Remove readme.txt with git rm

`git rm readme.txt` deleted the file from the working directory and staged the deletion (readme.txt disappears from the Explorer panel). I then committed it with `git commit -m "now removes readme"`, creating commit **7623bb5** (1 file changed, 0 insertions, 0 deletions, delete mode readme.txt).

Screenshot: git rm readme.txt.



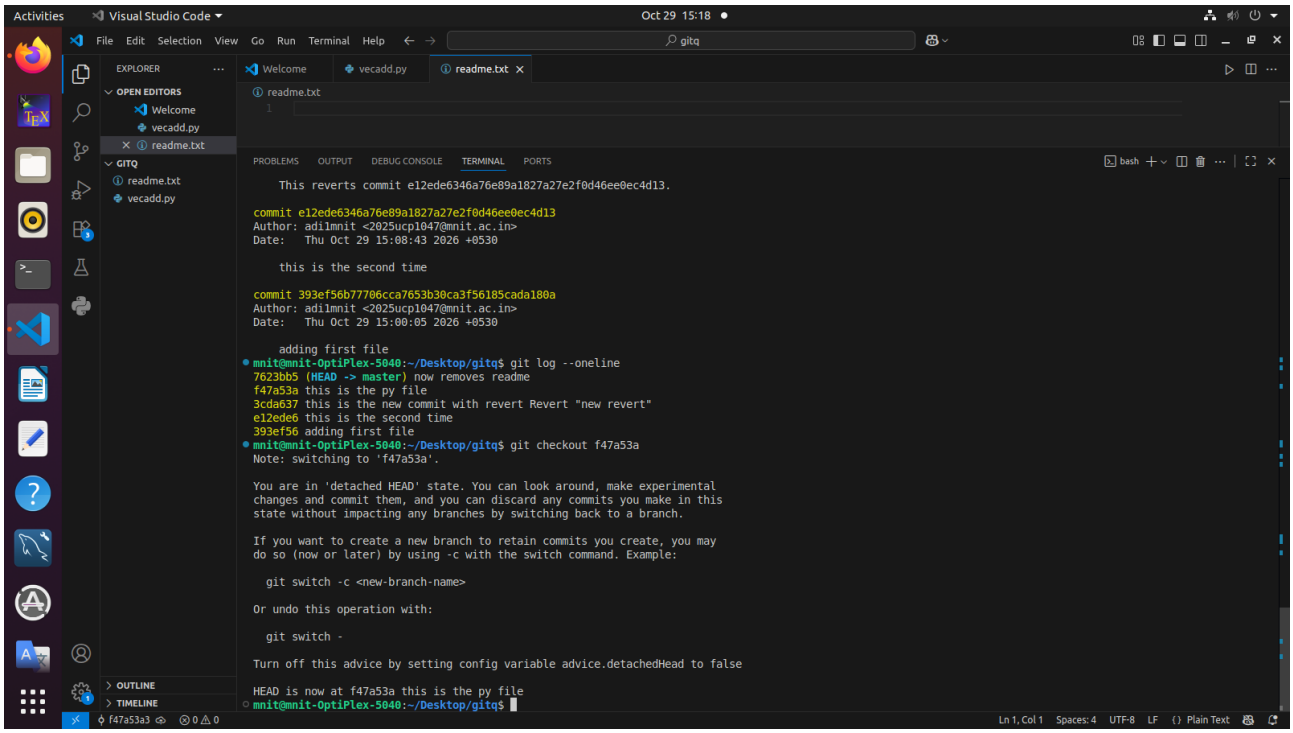
Screenshot: committing the removal.



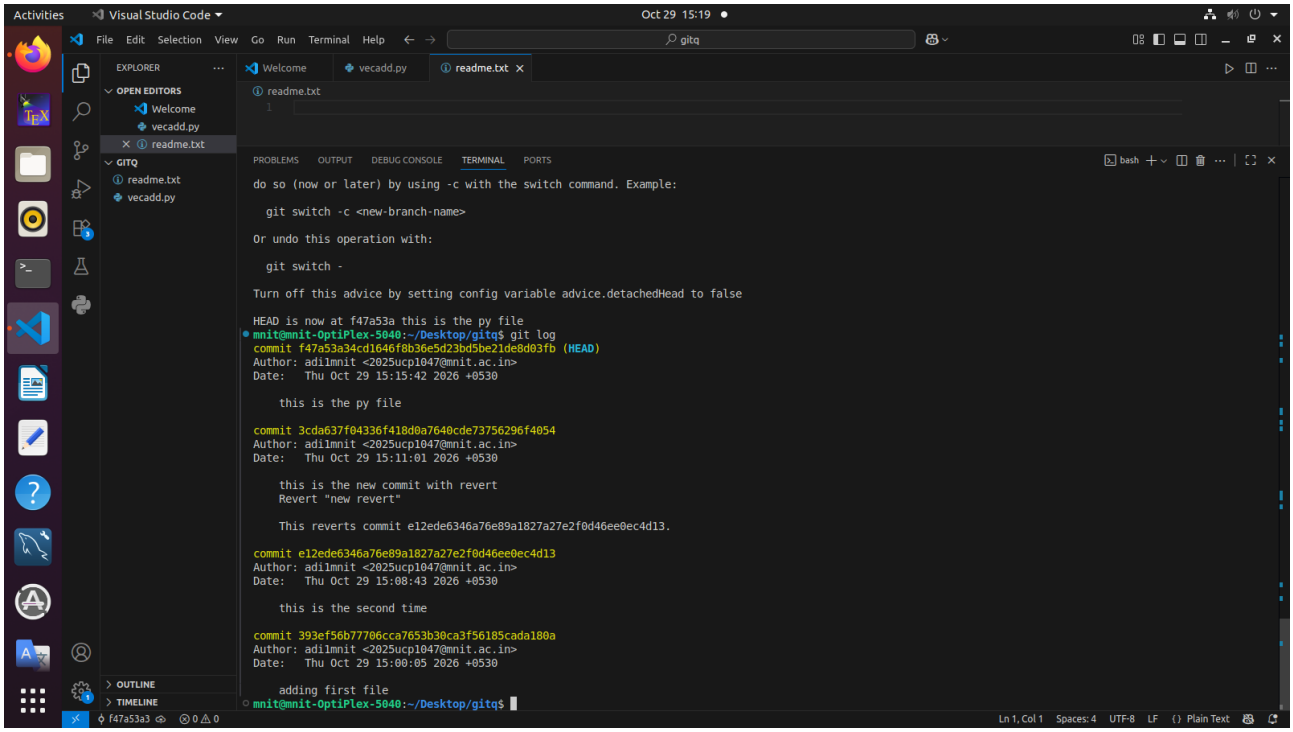
Step 11: Check out an earlier commit

`git log --oneline` gave the short hashes of all commits. I ran `git checkout f47a53a` and Git moved to a **detached HEAD** state at "this is the py file". The working directory now contains readme.txt again (as it was at that commit) together with vecadd.py, and git log only shows commits up to f47a53a, so the later "now removes readme" commit is not shown from here.

Screenshot: git log --oneline and git checkout f47a53a (detached HEAD message).



Screenshot: git log while in the detached HEAD state.



Observation: git checkout vs git revert

git revert keeps the full history and adds a new commit that cancels an earlier one, so it is safe for shared work. **git checkout <hash>** does not change the history at all; it only moves HEAD and the working directory to look like an older snapshot so I can inspect it. Commits made in detached HEAD can be lost unless a branch is created for them.

Step 12: Return to the latest commit

`git checkout main` failed twice with *"pathspec 'main' did not match any file(s)"* because my branch is named **master**, not main. `git checkout master` then worked (*Switched to branch 'master'*), moving HEAD back to the latest commit; readme.txt is gone again since it was removed in the last commit.

Screenshot: failed git checkout main and successful git checkout master.

